Announcements

1. Midterm coming
2. PS’es announced earlier (+1 day)
3. Extra day (+1 day)

Lecture 10
Representation Strategies for Data Types



T. METIN SEZGIN

The Environment Interface

- Environment
  - Function that maps variables to values

$$\{(var_1, val_1), \ldots, (var_n, val_n)\}$$

- The interface

$$\begin{aligned}
&\texttt{(empty-env)} && = \lceil \emptyset \rceil \\
&\texttt{(apply-env } \lceil f \rceil \ var\texttt{)} && = f(var) \\
&\texttt{(extend-env } var \ v \ \lceil f \rceil\texttt{)} && = \lceil g \rceil, \\
& && \text{where } g(var_1) = \begin{cases} v & \text{if } var_1 = var \\ f(var_1) & \text{otherwise} \end{cases}
\end{aligned}$$

Data Structure Representation

- The interface
  - Constructors
  - Observers

$$
\begin{aligned}
&\texttt{(empty-env)} && = \lceil \emptyset \rceil \\
&\texttt{(apply-env } \lceil f \rceil \ var\texttt{)} && = f(var) \\
&\texttt{(extend-env } var \ v \ \lceil f \rceil\texttt{)} && = \lceil g \rceil, \\
& && \text{where } g(var_1) = \begin{cases} v & \text{if } var_1 = var \\ f(var_1) & \text{otherwise} \end{cases}
\end{aligned}
$$

- For example

```
(define e
  (extend-env 'd 6
    (extend-env 'y 8
      (extend-env 'x 7
        (extend-env 'y 14
          (empty-env))))))
```

$e(\mathrm{d}) = 6, e(\mathrm{x}) = 7, e(\mathrm{y}) = 8$

- The grammar

Env-exp ::= `(empty-env)`
::= `(extend-env` *Identifier* *Scheme-value* *Env-exp*`)`

Implementation



Env = (empty-env) | (extend-env *Var* *SchemeVal* *Env*)
Var = *Sym*

empty-env : () → *Env*

```
(define empty-env
  (lambda () (list 'empty-env)))
```

extend-env : *Var* × *SchemeVal* × *Env* → *Env*

```
(define extend-env
  (lambda (var val env)
    (list 'extend-env var val env)))
```

apply-env : *Env* × *Var* → *SchemeVal*

```
(define apply-env
  (lambda (env search-var)
    (cond
      ((eqv? (car env) 'empty-env)
       (report-no-binding-found search-var))
      ((eqv? (car env) 'extend-env)
       (let ((saved-var (cadr env))
             (saved-val (caddr env))
             (saved-env (cadddr env)))
         (if (eqv? search-var saved-var)
           saved-val
           (apply-env saved-env search-var))))
      (else
        (report-invalid-env env)))))
```

Procedural Representation



$Env = Var \rightarrow SchemeVal$

empty-env : $() \rightarrow Env$

```
(define empty-env
  (lambda ()
    (lambda (search-var)
      (report-no-binding-found search-var))))
```

extend-env : $Var \times SchemeVal \times Env \rightarrow Env$

```
(define extend-env
  (lambda (saved-var saved-val saved-env)
    (lambda (search-var)
      (if (eqv? search-var saved-var)
        saved-val
        (apply-env saved-env search-var)))))
```

apply-env : $Env \times Var \rightarrow SchemeVal$

```
(define apply-env
  (lambda (env search-var)
    (env search-var)))
```

The general form of `define-datatype`



```
(define-datatype environment environment?
  (empty-env)
  (extend-env
    (bvar symbol?)
    (bval expval?)
    (saved-env environment?))
  (extend-env-rec
    (id symbol?)
    (bvar symbol?)
    (body expression?)
    (saved-env environment?)))
```

(define-datatype *type-name type-predicate-name*
{(*variant-name* {(*field-name predicate*)}*)}+)

Example uses of `define-datatype`

- Lets define a “triple” structure using racket

Depending on how you look at it, **Racket** is

- a *programming language*—a dialect of Lisp and a descendant of Scheme;

> See Dialects of Racket and Scheme for more information on other dialects of Lisp and how they relate to Racket.

- a *family* of programming languages—variants of Racket, and more; or
- a set of *tools*—for using a family of programming languages.

Where there is no room for confusion, we use simply *Racket*.

Racket's main tools are

- **racket**, the core compiler, interpreter, and run-time system;
- **DrRacket**, the programming environment; and
- **raco**, a command-line tool for executing **Ra**cket **co**mmands that install packages, build libraries, and more.

Example uses of `define-datatype`



S-list ::= ({*S-exp*}*)
S-exp ::= *Symbol* | *S-list*

```
(define-datatype s-list s-list?
  (empty-s-list)
  (non-empty-s-list
    (first s-exp?)
    (rest s-list?)))

(define-datatype s-exp s-exp?
  (symbol-s-exp
    (sym symbol?))
  (s-list-s-exp
    (slst s-list?)))
```

Nugget



We can represent any data structure easily using define-datatype

Lecture 11
Abstract Syntax, Representation, Interpretation



T. METIN SEZGIN

Nuggets of the lecture



- Syntax is all about structure
- Semantics is all about meaning
- We can use abstract syntax to represent programs as trees
- Parsing takes a program builds a syntax tree
- Unparsing converts abstract tree to a text file
- Big picture of compilers and interpreters

Human vs. the computer

- Lambda calculus

$$
\begin{aligned}
\mathit{LcExp} &::= \mathit{Identifier} \\
&::= \texttt{(lambda (}\mathit{Identifier}\texttt{)}\ \mathit{LcExp}\texttt{)} \\
&::= \texttt{(}\mathit{LcExp}\ \mathit{LcExp}\texttt{)}
\end{aligned}
$$

- Alternative syntax

$$
\begin{aligned}
\mathit{Lc\text{-}exp} &::= \mathit{Identifier} \\
&::= \texttt{proc}\ \mathit{Identifier}\ \texttt{=>}\ \mathit{Lc\text{-}exp} \\
&::= \mathit{Lc\text{-}exp}\ (\mathit{Lc\text{-}exp})
\end{aligned}
$$

- The computer

```
(define-datatype lc-exp lc-exp?
  (var-exp
    (var identifier?))
  (lambda-exp
    (bound-var identifier?)
    (body lc-exp?))
  (app-exp
    (rator lc-exp?)
    (rand lc-exp?)))
```

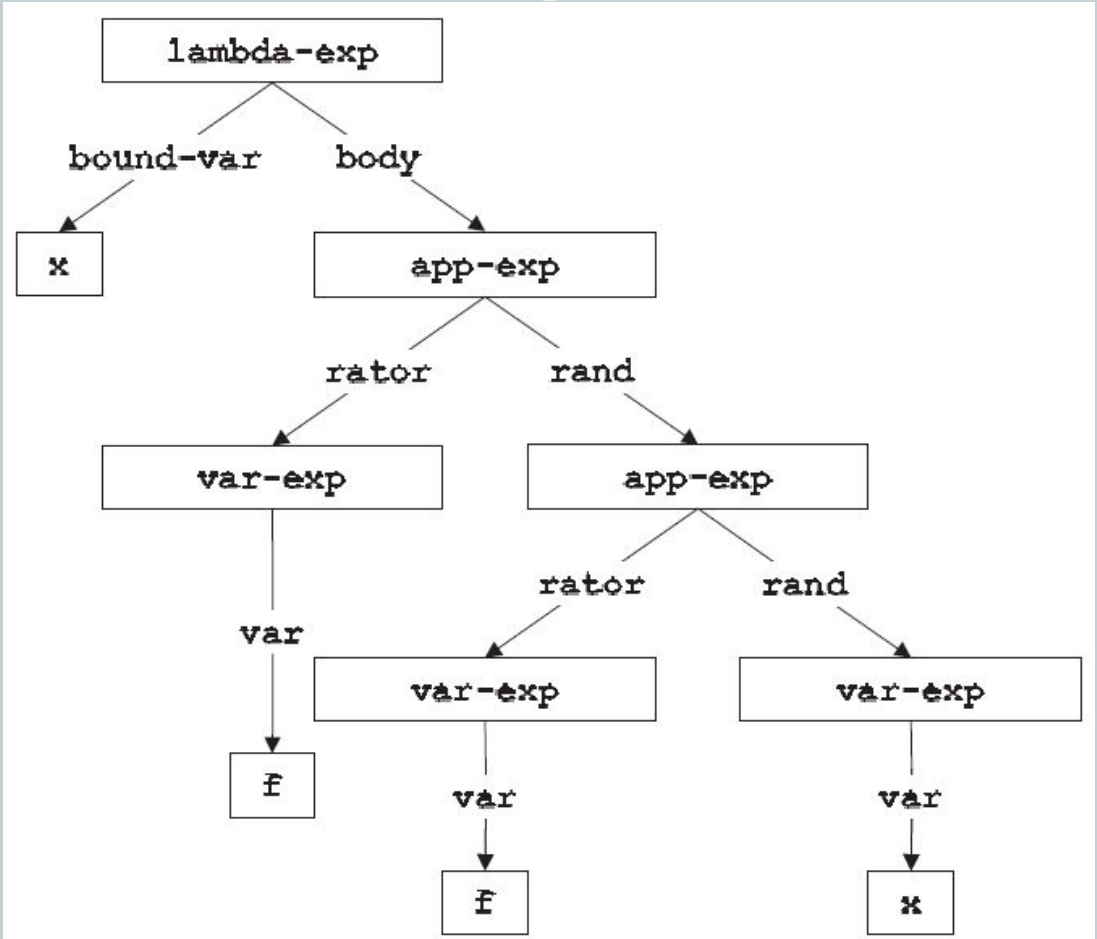
$$
\begin{aligned}
\mathit{Lc\text{-}exp} &::= \mathit{Identifier} \\
&\quad \boxed{\texttt{var-exp (var)}} \\
&::= \texttt{(lambda (}\mathit{Identifier}\texttt{)}\ \mathit{Lc\text{-}exp}\texttt{)} \\
&\quad \boxed{\texttt{lambda-exp (bound-var body)}} \\
&::= \texttt{(}\mathit{Lc\text{-}exp}\ \mathit{Lc\text{-}exp}\texttt{)} \\
&\quad \boxed{\texttt{app-exp (rator rand)}}
\end{aligned}
$$

Nugget



We can use abstract syntax to represent programs as trees

A specific example



Abstract syntax tree for `(lambda (x) (f (f x)))`

Nugget



Parsing takes a program builds a syntax tree

Parsing expressions



parse-expression : *SchemeVal* → *LcExp*

```
(define parse-expression
  (lambda (datum)
    (cond
      ((symbol? datum) (var-exp datum))
      ((pair? datum)
       (if (eqv? (car datum) 'lambda)
         (lambda-exp
           (car (cadr datum))
           (parse-expression (caddr datum)))
         (app-exp
           (parse-expression (car datum))
           (parse-expression (cadr datum)))))
      (else (report-invalid-concrete-syntax datum)))))
```

Nugget



Unparsing goes in the reverse direction

“Unparsing”



unparse-lc-exp : *LcExp* → *SchemeVal*

```
(define unparse-lc-exp
  (lambda (exp)
    (cases lc-exp exp
      (var-exp (var) var)
      (lambda-exp (bound-var body)
        (list 'lambda (list bound-var)
          (unparse-lc-exp body)))
      (app-exp (rator rand)
        (list
```

The next few weeks



- Expressions
- Binding of variables
- Scoping of variables
- Environment
- Interpreters

Nugget



Semantics is all about evaluating programs, finding their “value”

Notation

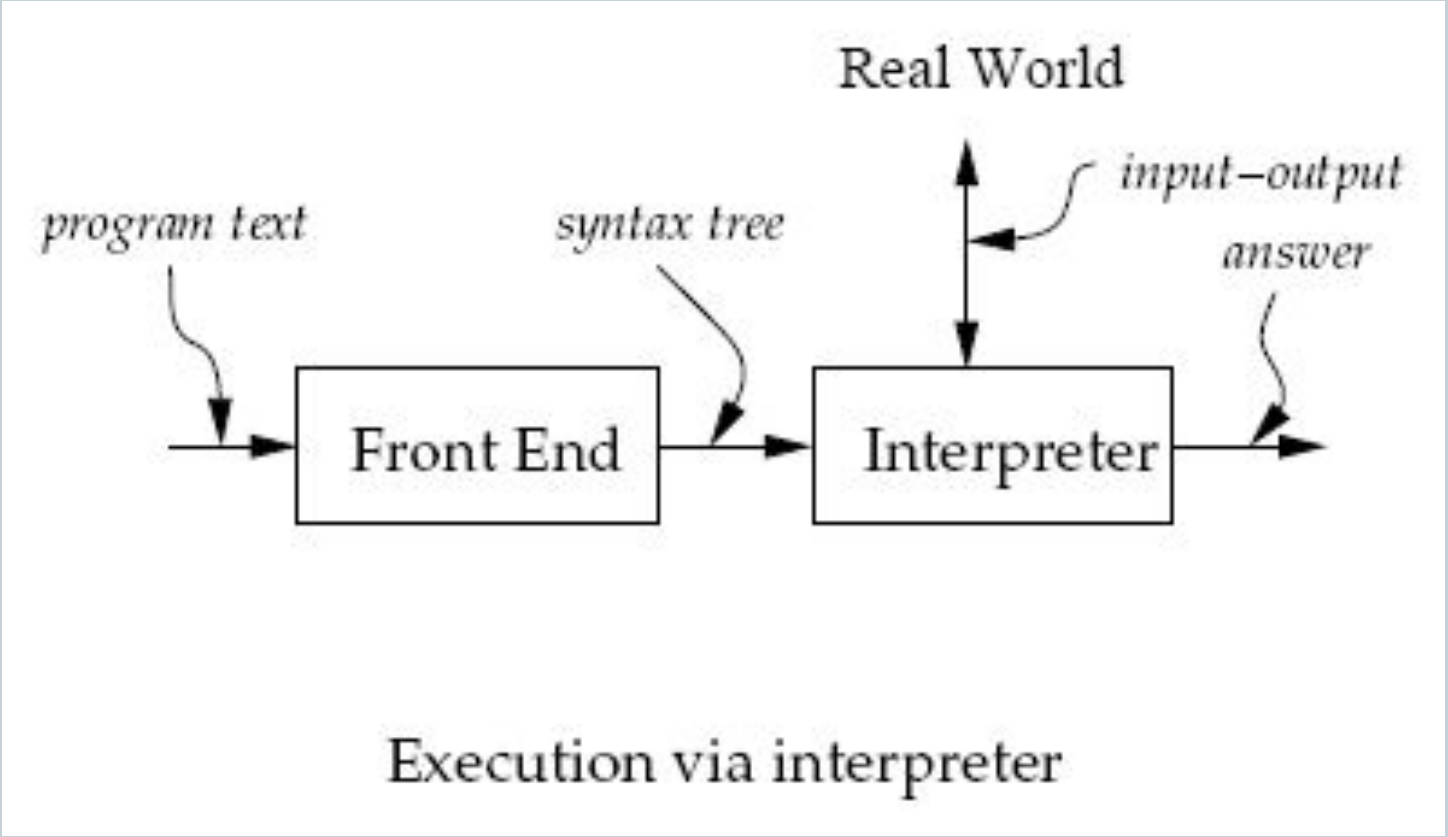


- Assertions for specification

$$(\texttt{value-of}\ \ exp\ \ \rho) = val$$

- Use rules from earlier chapters and specifications to compute values

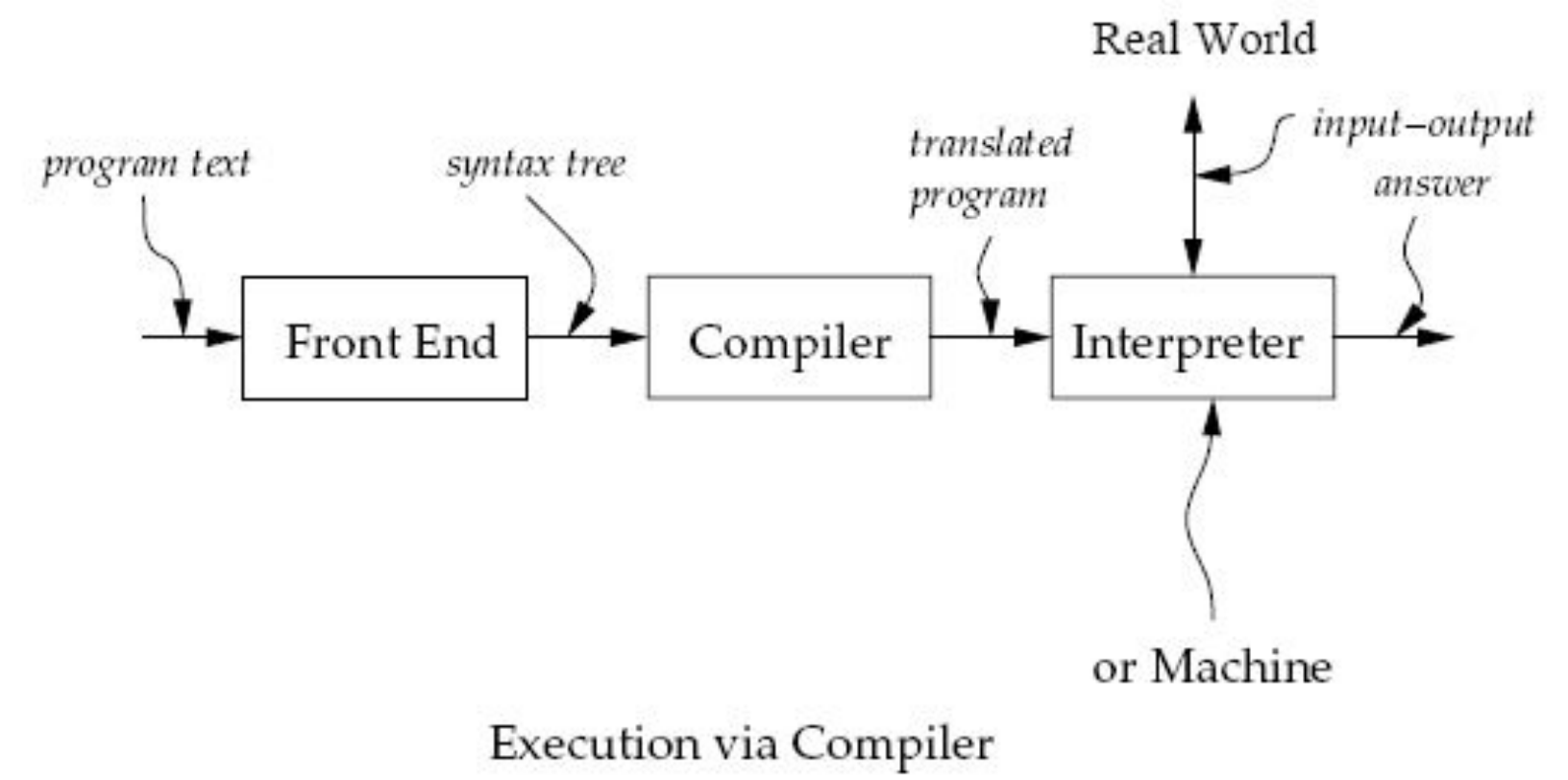
The big picture – interpreter



Execution via interpreter

Source language (defined language), implementation language (defining language), target language,

The big picture – compiler



Execution via Compiler

Source language (defined language), implementation language (defining language), target language, bytecode, virtual machine

About compilation

- Compilation
  - Analyzer
    - Scanning (lexical scanning)
      - Generates
        - Lexemes
        - Lexical items
        - Tokens
    - Parsing
      - Generates
        - AST
        - Syntactic structure
        - Grammatical structure
  - Translator
- All this work simplified
  - Lexical analyzers (lex)
  - Parser generators (yacc)
  - Use scheme ☺

```
int main()
{
        printf("hello, world");
        return 0;
}
```

Nugget



Evaluating programs, requires understanding the expressions of the language

LET: our pet language



Program ::= *Expression*

```
a-program (exp1)
```

Expression ::= *Number*

```
const-exp (num)
```

Expression ::= `-(`*Expression* `,` *Expression*`)`

```
diff-exp (exp1 exp2)
```

Expression ::= `zero?` `(`*Expression*`)`

```
zero?-exp (exp1)
```

Expression ::= `if` *Expression* `then` *Expression* `else` *Expression*

```
if-exp (exp1 exp2 exp3)
```

Expression ::= *Identifier*

```
var-exp (var)
```

Expression ::= `let` *Identifier* `=` *Expression* `in` *Expression*

```
let-exp (var exp1 body)
```

An example program

- Input

```
"-(55, -(x,11))"
```

- Scanning & parsing

```
(scan&parse "-(55, -(x,11))")
```

- The AST

```
#(struct:a-program
   #(struct:diff-exp
      #(struct:const-exp 55)
      #(struct:diff-exp
         #(struct:var-exp x)
         #(struct:const-exp 11))))
```

Program ::= *Expression*
`a-program (exp1)`

Expression ::= *Number*
`const-exp (num)`

Expression ::= `-(`*Expression* `,` *Expression*`)`
`diff-exp (exp1 exp2)`

Expression ::= `zero?` `(`*Expression*`)`
`zero?-exp (exp1)`

Expression ::= `if` *Expression* `then` *Expression* `else` *Expression*
`if-exp (exp1 exp2 exp3)`

Expression ::= *Identifier*
`var-exp (var)`

Expression ::= `let` *Identifier* `=` *Expression* `in` *Expression*
`let-exp (var exp1 body)`